📄 ARCHITECTURE_DECISIONS.md

> **Project:** Prism IQ
>
> **Version:** 1.0
>
> **Status:** Approved
>
> **Purpose:** Record all major architectural decisions and their rationale.

ADR-001 — Why Prism IQ?

Decision

Build Prism IQ as an **AI-assisted analytics workspace** instead of another dashboard application.

Reason

Most student projects stop after:

- Upload
- Charts
- Download

Prism IQ focuses on the complete analytics journey.

```
Import

↓

Profile

↓

Clean

↓

Explore

↓

Predict
```

The product solves a workflow instead of a single task.

Benefits

- Better UX
- Better scalability
- Better portfolio value
- Clear product identity

ADR-002 — Pipeline Architecture

Decision

The application follows a **pipeline-first architecture**.

Reason

Every dataset naturally follows sequential processing.

```
Raw

↓

Validated

↓

Profiled

↓

Cleaned

↓

Analyzed
```

Instead of isolated modules, every stage builds upon the previous one.

Benefits

- Easier debugging
- Clear progress tracking

- Simpler frontend navigation
- Predictable state management

ADR-003 — FastAPI

Decision

Use FastAPI.

Alternatives Considered

- Flask
- Django
- Node.js (Express)
- NestJS

Why FastAPI?

- Native async support
- Automatic OpenAPI documentation
- Excellent type safety
- Pydantic validation
- High performance
- Strong Python ecosystem

Benefits

- Faster development
- Better documentation
- Easier testing
- Modern architecture

ADR-004 — PostgreSQL

Decision

Use PostgreSQL.

Alternatives

- SQLite
- MongoDB
- MySQL

Why PostgreSQL?

- JSONB support
- Strong relational model
- Excellent SQLAlchemy integration
- Reliable indexing
- Mature ecosystem

Benefits

- Flexible metadata storage
- High performance
- Future scalability

ADR-005 — Store Metadata Only

Decision

Do **not** store uploaded datasets inside PostgreSQL.

Store

- Metadata
- Status
- Reports
- Model references
- Analysis metadata

Do Not Store

- CSV
- Excel
- PDF
- Trained models

Reason

Large binary files increase database size unnecessarily.

Filesystem storage is simpler and more efficient for the MVP.

Benefits

- Smaller database

- Faster backups
- Easier file management

ADR-006 — Repository Pattern

Decision

Use layered architecture.

```
Router

↓

Service

↓

Repository

↓

Database
```

Reason

Separate business logic from persistence logic.

Benefits

- Testability
- Maintainability
- Easier refactoring

ADR-007 — React + TypeScript

Decision

Frontend uses React with TypeScript.

Reason

Type safety reduces runtime bugs.

Component architecture fits reusable dashboards.

Benefits

- Better DX
- Cleaner components
- Strong ecosystem

ADR-008 — Tailwind CSS + shadcn/ui

Decision

Use Tailwind CSS and shadcn/ui.

Alternatives

- Material UI
- Bootstrap
- Ant Design

Reason

We want complete visual control and a premium SaaS aesthetic.

Benefits

- Faster UI development
- Consistent design
- Highly customizable

ADR-009 — Dark Mode First

Decision

The MVP launches with a dark-first interface.

Reason

Analytics dashboards are used for long sessions.

Dark mode reduces visual fatigue and aligns with modern SaaS products.

Future

Light mode may be added later.

ADR-010 — Workspace Navigation

Decision

Navigation follows pipeline stages rather than feature categories.

```
Import

↓

Profile

↓

Clean

↓

Explore

↓

Predict
```

Reason

This mirrors how users think about data analysis.

Benefits

- Better onboarding
- Guided workflow
- Lower cognitive load

ADR-011 — Machine Learning

Decision

Use Scikit-learn.

Alternatives

- TensorFlow
- PyTorch

Reason

Prism IQ focuses on structured tabular datasets.

Scikit-learn provides robust classical ML algorithms with minimal complexity.

Benefits

- Faster implementation
- Easier interpretation
- Stable ecosystem

ADR-012 — Automatic Model Selection

Decision

Train multiple supported models and present the best-performing one.

Reason

Most users cannot choose algorithms confidently.

The platform simplifies the decision.

Benefits

- Better UX
- Reduced manual experimentation

ADR-013 — Rule-Based Recommendations

Decision

The MVP uses a deterministic rule engine instead of an LLM.

Reason

- Predictable outputs
- No API cost
- No latency
- Easier testing
- Works offline

Future

LLM-powered insights can be introduced later as an optional enhancement.

ADR-014 — Local File Storage

Decision

Artifacts are stored on the local filesystem during the MVP.

Future

Potential migration to cloud object storage (e.g., S3-compatible services) without changing higher-level business logic.

Benefits

- Simplicity
- Faster setup
- Lower cost

ADR-015 — API Style

Decision

Use REST APIs.

Reason

- Simple
- Predictable
- Native FastAPI support
- Easy frontend integration

Future

GraphQL is not required for current requirements.

ADR-016 — Report Generation

Decision

Generate concise executive reports.

Reason

Long reports are rarely read.

Focus on:

- Summary
- Key insights
- Recommendations
- Important charts

Benefits

Higher practical value for users.

ADR-017 — Chart Libraries

Decision

Use Plotly + Recharts.

Reason

Plotly supports advanced interactivity.

Recharts is lightweight for dashboard KPIs and standard charts.

Benefits

- Flexible visualization
- Modern UI
- Better user experience

ADR-018 — Backend Processing

Decision

Run processing synchronously for the MVP.

Reason

The current scope and deadline do not justify the complexity of distributed task queues.

Future

Background workers (Celery + Redis) can be introduced when processing larger datasets or supporting concurrent users.

ADR-019 — UUIDs

Decision

Use UUIDs as primary keys.

Reason

- Safer identifiers

- Better for future APIs
- Avoid predictable IDs

Benefits

- Security
- Scalability

ADR-020 — Documentation First

Decision

Complete architecture and documentation before implementation.

Reason

Clear documentation reduces ambiguity and improves AI-assisted development consistency.

Benefits

- Faster implementation
- Fewer architectural changes
- Better maintainability
- Easier onboarding

ADR-021 — AI-Assisted Development

Decision

AI tools (such as Antigravity and Ponytail) are treated as engineering assistants, not autonomous decision-makers.

Rules

- AI must follow the documented architecture.
- AI should not redesign the system without explicit approval.
- Human review is required for architectural changes.
- Code should be production-oriented, not just "working."

ADR-022 — Scope Management

Decision

Prioritize a polished MVP over a feature-heavy prototype.

Included

- Import
- Profiling
- Cleaning
- EDA
- ML
- Recommendations
- Reports

Deferred

- Authentication
- Teams
- Cloud storage
- Background jobs
- LLM integrations
- Forecasting
- Real-time collaboration

ADR-023 — Engineering Philosophy

Prism IQ should always optimize for:

1. Simplicity over unnecessary complexity.
2. Readability over cleverness.
3. Maintainability over shortcuts.
4. Consistency over personal preference.
5. User value over feature count.

📌 Architecture Review Process

Any future architectural change must answer these questions:

1. **What problem does this solve?**
2. **Why is the current approach insufficient?**
3. **What alternatives were considered?**
4. **What trade-offs are introduced?**

5. **Does it align with Prism IQ's core philosophy?**

Only if these questions are satisfactorily answered should an ADR be updated or a new one added.

🏁 Final Summary

This document freezes the architectural direction of Prism IQ. Every major decision is documented with its rationale, allowing future contributors—or even your future self—to understand **why** the system was designed this way, not just **how** it works.
